Arithmetic Coding and Range Coding: A Detailed Tutorial

1 Arithmetic Coding and Range Coding: A Detailed Tutorial

1.1 Introduction

Entropy coding is the final stage in many compression systems. Instead of assigning fixed codewords to symbols (like Huffman coding), **Arithmetic Coding** and **Range Coding** represent the entire message as a single number inside the unit interval $[0, 1)$. This allows code lengths to approach the theoretical entropy limit.

Key Idea

Arithmetic coding does not assign codes to symbols individually. Instead, it assigns a code to the entire *sequence* by repeatedly shrinking an interval. The more probable the message, the larger the final interval and the fewer bits needed.

We now explain arithmetic coding in detail, followed by a complete worked example, a comparison with range coding, and finally the same example solved using range coding.

1.2 Arithmetic Coding: Theory and Algorithm

Arithmetic coding encodes a message by repeatedly narrowing an interval in $[0, 1)$. Each symbol reduces the interval based on its probability.

Algorithm: Arithmetic Coding Encoding

Step 0: Initialization
Set low $= 0$ and high $= 1$.

Step 1: Compute cumulative frequencies
Given an alphabet $\{s_1, s_2, \ldots, s_k\}$ with probabilities $p(s_i)$, compute:

$$C(s_1) = 0, \qquad C(s_i) = \sum_{j<i} p(s_j) \text{ for } i = 2, 3, \ldots, k$$

Step 2: For each symbol in the message to be encoded
For each symbol s (taken in order from the message), perform:

$$\begin{aligned} \text{range} &= \text{high} - \text{low} \\ \text{high} &= \text{low} + \text{range} \times (C(s) + p(s)) \\ \text{low} &= \text{low} + \text{range} \times C(s) \end{aligned}$$

Step 3: Generate the final code
After encoding all symbols, any number x in the interval $[\text{low}, \text{high})$ represents the entire message. Typically, we output:

$$\text{code} = \frac{\text{low} + \text{high}}{2} \quad \text{(the midpoint of the final interval)}$$

1.3 Example: Arithmetic Coding Step-by-Step

Consider the alphabet $\{A, B, C\}$ with frequencies:

$$f(A) = 5, \qquad f(B) = 3, \qquad f(C) = 2, \qquad \text{Total} = 10.$$

Thus probabilities:

$$p(A) = 0.5, \qquad p(B) = 0.3, \qquad p(C) = 0.2.$$

We want to encode the message:

$$\text{MESSAGE} = \texttt{BAC}$$

Step 0: Initialization

$$\text{low} = 0, \qquad \text{high} = 1$$

Step 1: Compute cumulative frequencies

The cumulative frequencies define the sub-interval boundaries and are sorted in alphabetical order:

$$C(A) = 0, \qquad C(B) = p(A) = 0.5, \qquad C(C) = p(A) + p(B) = 0.8$$

Step 2: Encode each symbol in the message

Symbol 1: B

$$\begin{aligned}
\text{range} &= \text{high} - \text{low} = 1 - 0 = 1 \\
\text{high} &= \text{low} + \text{range} \times (C(B) + p(B)) = 0 + 1 \times (0.5 + 0.3) = 0.8 \\
\text{low} &= \text{low} + \text{range} \times C(B) = 0 + 1 \times 0.5 = 0.5
\end{aligned}$$

Interval after encoding B: $[0.5,\ 0.8)$

Symbol 2: A

$$\begin{aligned}
\text{range} &= \text{high} - \text{low} = 0.8 - 0.5 = 0.3 \\
\text{high} &= \text{low} + \text{range} \times (C(A) + p(A)) = 0.5 + 0.3 \times (0 + 0.5) = 0.5 + 0.15 = 0.65 \\
\text{low} &= \text{low} + \text{range} \times C(A) = 0.5 + 0.3 \times 0 = 0.5
\end{aligned}$$

Interval after encoding A: $[0.5,\ 0.65)$

Symbol 3: C

$$\begin{aligned}
\text{range} &= \text{high} - \text{low} = 0.65 - 0.5 = 0.15 \\
\text{high} &= \text{low} + \text{range} \times (C(C) + p(C)) = 0.5 + 0.15 \times (0.8 + 0.2) = 0.5 + 0.15 \times 1.0 = 0.65 \\
\text{low} &= \text{low} + \text{range} \times C(C) = 0.5 + 0.15 \times 0.8 = 0.5 + 0.12 = 0.62
\end{aligned}$$

Final interval after encoding all symbols: $[0.62,\ 0.65)$

Step 3: Generate the final code

Any number in $[0.62, 0.65)$ encodes the message `BAC`. We choose the midpoint:

$$\text{code} = \frac{0.62 + 0.65}{2} = 0.635$$

This is the arithmetic-coded representation of `BAC`. In practice, we would output this number in binary form.

1.4 Why People Prefer Range Coding

Arithmetic coding is theoretically elegant but has practical implementation challenges:

- **Arbitrary precision requirement:** As encoding progresses, the interval becomes very small. To represent the final interval precisely, we need high-precision arithmetic. For long messages, this requires arbitrary-precision numbers or specialized libraries.
- **Incremental output complexity:** In practice, we don't want to wait until the entire message is encoded before outputting bits. Instead, we output bits incrementally as they become fixed. This process—often called *normalization* or *bit-stuffing*—requires careful handling of carry propagation.

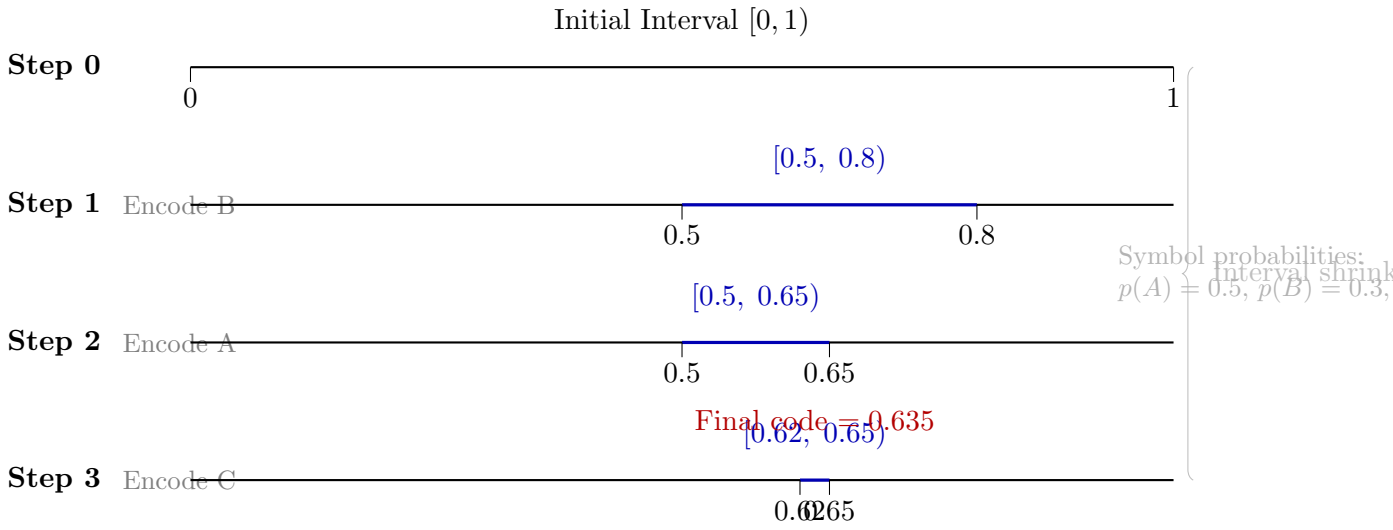

Figure 1: Arithmetic Coding Interval Refinement for Message `BAC`. Each step selects a sub-interval based on the symbol being encoded. The final code 0.635 is the midpoint of the final interval $[0.62, 0.65)$.

- **Computational cost:** Multiplication and division operations at each step are more expensive than the integer operations used in range coding.
- **Historical patent issues:** Arithmetic coding was subject to patents for many years, discouraging its use in some applications (though this is no longer an issue today).

Normalization in Arithmetic Coding

Unlike floating-point numbers that can underflow to zero, the interval in arithmetic coding never underflows—it simply becomes very small. The term *normalization* in arithmetic coding refers to the process of **outputting bits incrementally** as they become known, not to preventing precision loss.

For example, after encoding the first few symbols, the interval $[0.5, 0.8)$ tells us that the first bit of the output will be `1` (since the entire interval lies in the top half of $[0, 1)$). We can output that bit immediately and double the interval to continue encoding.

If we are content to store the entire final fraction (like 0.635) without incremental output, no normalization is needed. However, for long messages, storing the full precision fraction becomes impractical, which is why incremental output is used in real implementations.

Range Coding addresses these challenges:

- Uses fixed-precision integer arithmetic (e.g., 32-bit or 64-bit integers)
- Handles incremental output naturally through renormalization
- Uses only integer operations (division, multiplication, shifts)
- Faster on most hardware architectures

Range coding is simply a **reparameterization of arithmetic coding** designed for efficient implementation.

1.5 Range Coding: Theory and Algorithm

Range coding works on integers rather than real-number intervals. The fundamental insight is that we can scale the $[0, 1)$ interval to a large integer range, e.g., $[0, 2^{32})$. All operations then become integer arithmetic.

Algorithm: Range Coding Encoding

Step 0: Initialization
Set low $= 0$ and range $= R$ (a large integer, typically 2^{32}).

Step 1: Compute cumulative frequencies
Given an alphabet $\{s_1, s_2, \ldots, s_k\}$ with frequencies $f(s_i)$ and total frequency total $= \sum_i f(s_i)$, compute:

$$C(s_1) = 0, \qquad C(s_i) = \sum_{j<i} f(s_j) \text{ for } i = 2, 3, \ldots, k$$

Step 2: For each symbol in the message to be encoded
For each symbol s (taken in order from the message), perform:

1. Compute unit = range/total (integer division)
2. Update low = low + unit $\times C(s)$
3. Update range = unit $\times f(s)$
4. **Renormalization:** While range $< 2^{24}$ (or other threshold), output the top byte of low and shift low and range left by 8 bits.

Step 3: Generate the final code
After encoding all symbols, output the remaining bytes of low to complete the compressed representation.

1.6 Range Coding Example: Same Message (BAC)

We now encode the same message `BAC` using integer range coding. We'll use a smaller initial range of $R = 2^{16} = 65536$ for simplicity in our calculations. The process is identical with 2^{32}.

Step 0: Initialization

$$\text{low} = 0, \qquad \text{range} = 65536$$

Step 1: Compute cumulative frequencies

$$f(A) = 5, \quad f(B) = 3, \quad f(C) = 2, \quad \text{total} = 10$$
$$C(A) = 0, \quad C(B) = 5, \quad C(C) = 8$$

Step 2: Encode each symbol in the message

Symbol 1: B

$$\begin{aligned}\text{unit} &= \frac{65536}{10} = 6553 \quad \text{(integer division truncates)}\\ \text{low} &= 0 + 6553 \times 5 = 32765\\ \text{range} &= 6553 \times 3 = 19659\end{aligned}$$

Renormalization check: $19659 \geq 2^{14}$? No renormalization needed for this simple example.

Real-number equivalent: $\left[\frac{32765}{65536}, \frac{32765+19659}{65536}\right) \approx [0.5000, 0.8000)$

Symbol 2: A

$$\begin{aligned}\text{unit} &= \frac{19659}{10} = 1965\\ \text{low} &= 32765 + 1965 \times 0 = 32765\\ \text{range} &= 1965 \times 5 = 9825\end{aligned}$$

Renormalization check: $9825 \geq 2^{14}$? No renormalization needed.

Real-number equivalent: $\left[\frac{32765}{65536}, \frac{32765+9825}{65536}\right) \approx [0.5000, 0.6500)$

Symbol 3: C

$$\begin{aligned}\text{unit} &= \frac{9825}{10} = 982\\ \text{low} &= 32765 + 982 \times 8 = 32765 + 7856 = 40621\\ \text{range} &= 982 \times 2 = 1964\end{aligned}$$

Renormalization check: $1964 \geq 2^{14}$? No renormalization needed.

Real-number equivalent: $\left[\frac{40621}{65536}, \frac{40621+1964}{65536}\right) \approx [0.6198, 0.6498)$

Step 3: Generate the final code

The final integer interval is $[\text{low}, \text{low} + \text{range}) = [40621, 42585)$. Any integer in this range encodes the message. We output the value 40621 as our compressed representation.

1.7 Range Coding Example with Renormalization

Now we present a more realistic example that demonstrates renormalization in action. We encode a longer message that will cause the range to become small enough to trigger byte output.

Consider the same alphabet $\{A, B, C\}$ with frequencies:

$$f(A) = 5, \quad f(B) = 3, \quad f(C) = 2, \quad \text{total} = 10.$$

Cumulative frequencies:

$$C(A) = 0, \quad C(B) = 5, \quad C(C) = 8.$$

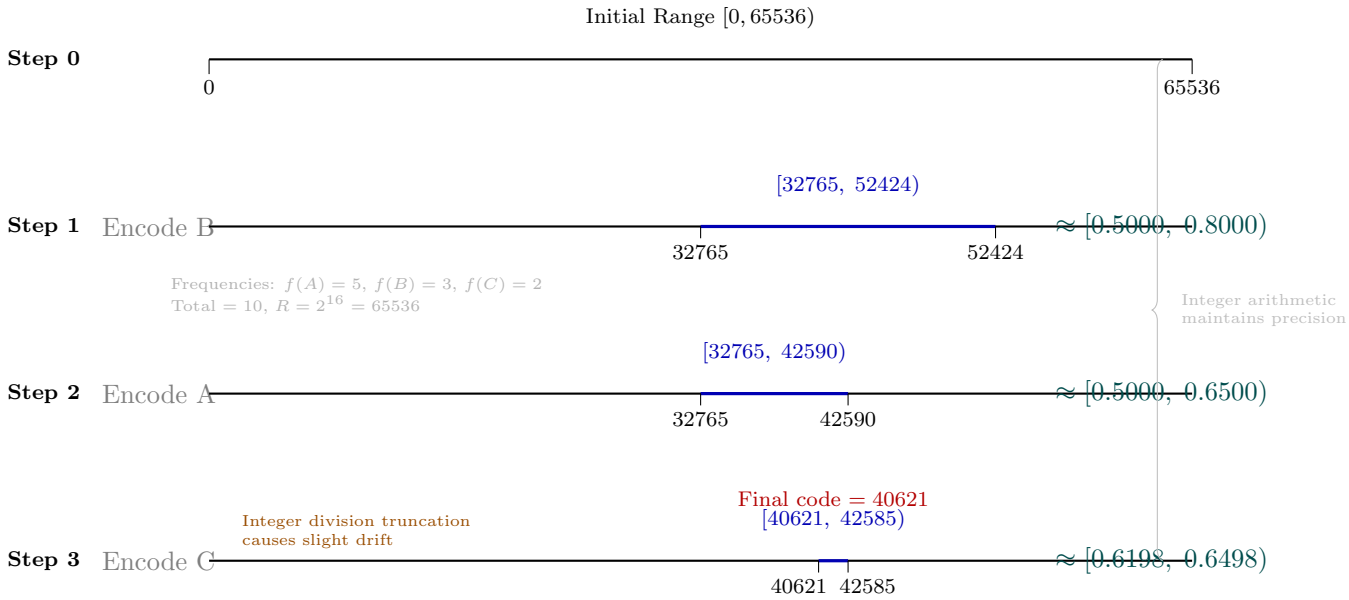

Figure 2: Range Coding Integer-Interval Refinement for Message `BAC`. Real-number equivalents are shown to the right.

We now encode a longer message:

$$\text{MESSAGE} = \texttt{B A C C C C}$$

Step 0: Initialization

$$\text{low} = 0, \qquad \text{range} = 2^{32} = 4{,}294{,}967{,}296$$

Step 1: Compute cumulative frequencies

$$C(A) = 0, \quad C(B) = 5, \quad C(C) = 8, \quad \text{total} = 10$$

Step 2: Encode each symbol in the message

Symbol 1: B

$$\begin{aligned}
\text{unit} &= \frac{4{,}294{,}967{,}296}{10} = 429{,}496{,}729 \\
\text{low} &= 0 + 429{,}496{,}729 \times 5 = 2{,}147{,}483{,}645 \\
\text{range} &= 429{,}496{,}729 \times 3 = 1{,}288{,}490{,}187
\end{aligned}$$

Renormalization check: $\text{range} = 1{,}288{,}490{,}187 \geq 2^{24} = 16{,}777{,}216$, so continue.

Symbol 2: A

$$\begin{aligned}
\text{unit} &= \frac{1{,}288{,}490{,}187}{10} = 128{,}849{,}018 \\
\text{low} &= 2{,}147{,}483{,}645 + 128{,}849{,}018 \times 0 = 2{,}147{,}483{,}645 \\
\text{range} &= 128{,}849{,}018 \times 5 = 644{,}245{,}090
\end{aligned}$$

Renormalization check: range = 644,245,090 $\geq$ 16,777,216, so continue.

Symbol 3: C

$$\text{unit} = \frac{644{,}245{,}090}{10} = 64{,}424{,}509$$
$$\text{low} = 2{,}147{,}483{,}645 + 64{,}424{,}509 \times 8 = 2{,}147{,}483{,}645 + 515{,}396{,}072 = 2{,}662{,}879{,}717$$
$$\text{range} = 64{,}424{,}509 \times 2 = 128{,}849{,}018$$

Renormalization check: range = 128,849,018 $\geq$ 16,777,216, so continue.

Symbol 4: C

$$\text{unit} = \frac{128{,}849{,}018}{10} = 12{,}884{,}901$$
$$\text{low} = 2{,}662{,}879{,}717 + 12{,}884{,}901 \times 8 = 2{,}662{,}879{,}717 + 103{,}079{,}208 = 2{,}765{,}958{,}925$$
$$\text{range} = 12{,}884{,}901 \times 2 = 25{,}769{,}802$$

Renormalization check: range = 25,769,802 $\geq$ 16,777,216, so continue.

Symbol 5: C

$$\text{unit} = \frac{25{,}769{,}802}{10} = 2{,}576{,}980$$
$$\text{low} = 2{,}765{,}958{,}925 + 2{,}576{,}980 \times 8 = 2{,}765{,}958{,}925 + 20{,}615{,}840 = 2{,}786{,}574{,}765$$
$$\text{range} = 2{,}576{,}980 \times 2 = 5{,}153{,}960$$

Renormalization check: range = 5,153,960 $<$ 16,777,216 — **renormalization triggered!**

Renormalization (after Symbol 5)

We output bytes until range $\geq 2^{24}$.

Iteration 1:

$$\text{output byte} = \text{low} \gg 24 = 2{,}786{,}574{,}765 \gg 24 = 166 \quad (\text{0xA6})$$
$$\text{low} = (\text{low} \ll 8)\ \&\ (2^{32} - 1) = 439{,}463{,}680$$
$$\text{range} = \text{range} \ll 8 = 5{,}153{,}960 \times 256 = 1{,}319{,}413{,}760$$

Now range = 1,319,413,760 $\geq$ 16,777,216, so stop renormalization.
Output byte: 0xA6 (166).

Symbol 6: C

$$\text{unit} = \frac{1{,}319{,}413{,}760}{10} = 131{,}941{,}376$$
$$\text{low} = 439{,}463{,}680 + 131{,}941{,}376 \times 8 = 439{,}463{,}680 + 1{,}055{,}531{,}008 = 1{,}494{,}994{,}688$$
$$\text{range} = 131{,}941{,}376 \times 2 = 263{,}882{,}752$$

Renormalization check: range = 263,882,752 $\geq$ 16,777,216, so continue.

Step 3: Generate the final code

After encoding all symbols, output the remaining bytes of low.
low = 1,494,994,688 in hexadecimal: 0x591C0000
The final output bytes are: $0x59, 0x1C, 0x00, 0x00$.

Complete Encoded Output

The compressed representation consists of:

- The byte output during renormalization: 0xA6
- The final low bytes: $0x59, 0x1C, 0x00, 0x00$

Thus the encoded stream is: $[166, 89, 28, 0, 0]$ (5 bytes total).
In hexadecimal: 0xA6 0x59 0x1C 0x00 0x00.

Encoding State Table

Step	Symbol	Low	Range	Output
Start	—	0	4,294,967,296	—
1	B	2,147,483,645	1,288,490,187	—
2	A	2,147,483,645	644,245,090	—
3	C	2,662,879,717	128,849,018	—
4	C	2,765,958,925	25,769,802	—
5	C	2,786,574,765	5,153,960	—
Renorm	—	439,463,680	1,319,413,760	166
6	C	1,494,994,688	263,882,752	—

Why This Sequence Triggers Renormalization

Four consecutive C's (each with frequency $2/10 = 0.2$) cause the range to shrink by a factor of 0.2 each time:

$$\begin{aligned} 644{,}245{,}090 \times 0.2 &= 128{,}849{,}018 \\ 128{,}849{,}018 \times 0.2 &= 25{,}769{,}802 \\ 25{,}769{,}802 \times 0.2 &= 5{,}153{,}960 < 16{,}777{,}216 \end{aligned}$$

This demonstrates how range coding handles interval narrowing through renormalization, outputting bytes as they become fixed.

1.8 Range Coding: Decoding Algorithm

Decoding is the inverse process. Given the compressed stream, we reconstruct the original message by determining which symbol's interval contains the current code value.

Algorithm: Range Coding Decoding

Step 0: Initialization
Set low $= 0$ and range $= R$ (same as encoder). Read the first 4 bytes from the compressed stream to form the initial code.

Step 1: Compute cumulative frequencies
Same as encoder: $C(s_1) = 0$, $C(s_i) = \sum_{j<i} f(s_j)$, and total $= \sum_i f(s_i)$.

Step 2: For each symbol to be decoded
For each symbol position in the message (until we have decoded all symbols), perform:

1. Compute the offset within the current interval:

$$\text{offset} = \frac{\text{code} - \text{low}}{\text{range}}$$

2. Find the symbol s such that:

$$\frac{C(s)}{\text{total}} \leq \text{offset} < \frac{C(s) + f(s)}{\text{total}}$$

   (Using integer arithmetic, find s where $C(s) \leq \frac{(\text{code}-\text{low})\times\text{total}}{\text{range}} < C(s) + f(s)$)

3. Output symbol s

4. Update low and range using the same formulas as the encoder:

$$\begin{aligned} \text{unit} &= \text{range}/\text{total} \\ \text{low} &= \text{low} + \text{unit} \times C(s) \\ \text{range} &= \text{unit} \times f(s) \end{aligned}$$

5. **Renormalization:** While range $< 2^{24}$, read the next byte from the stream and update:

$$\begin{aligned} \text{low} &= (\text{low} \ll 8) \ \& \ (2^{32} - 1) \\ \text{code} &= ((\text{code} \ll 8) \ \& \ (2^{32} - 1)) \mid \text{next_byte} \\ \text{range} &= \text{range} \ll 8 \end{aligned}$$

1.9 Range Coding Decoding Example

We now decode the compressed stream produced in the previous normalization example. The encoded stream was:

$$[166, 89, 28, 0, 0] \quad \text{in decimal, or} \quad [\text{0xA6}, \text{0x59}, \text{0x1C}, \text{0x00}, \text{0x00}] \text{ in hex}$$

We know the original message was `B A C C C C` (6 symbols). We will decode it to verify.

Step 0: Initialization

$$\text{low} = 0, \qquad \text{range} = 2^{32} = 4{,}294{,}967{,}296$$

Read the first 4 bytes to form the initial code:

$$\text{code} = \text{0xA6591C00} = 2{,}789{,}021{,}696$$

Step 1: Compute cumulative frequencies

$$f(A) = 5, \quad f(B) = 3, \quad f(C) = 2, \quad \text{total} = 10$$
$$C(A) = 0, \quad C(B) = 5, \quad C(C) = 8$$

Step 2: Decode each symbol

Symbol 1:

Compute offset:

$$\text{offset} = \frac{\text{code} - \text{low}}{\text{range}} = \frac{2{,}789{,}021{,}696}{4{,}294{,}967{,}296} \approx 0.6494$$

Find symbol containing 0.6494:

- A: $[0, 0.5)$ — no
- B: $[0.5, 0.8)$ — yes, contains 0.6494
- C: $[0.8, 1.0)$ — no

Decoded symbol: **B**

Update using B ($C(B) = 5$, $f(B) = 3$):

$$\text{unit} = \frac{4{,}294{,}967{,}296}{10} = 429{,}496{,}729$$
$$\text{low} = 0 + 429{,}496{,}729 \times 5 = 2{,}147{,}483{,}645$$
$$\text{range} = 429{,}496{,}729 \times 3 = 1{,}288{,}490{,}187$$

Renormalization check: range $= 1{,}288{,}490{,}187 \geq 16{,}777{,}216$, so continue.

Symbol 2:

Compute offset:

$$\text{offset} = \frac{2{,}789{,}021{,}696 - 2{,}147{,}483{,}645}{1{,}288{,}490{,}187} = \frac{641{,}538{,}051}{1{,}288{,}490{,}187} \approx 0.4980$$

Find symbol containing 0.4980:

- A: $[0, 0.5)$ — yes, contains 0.4980
- B: $[0.5, 0.8)$ — no
- C: $[0.8, 1.0)$ — no

Decoded symbol: **A**
Update using A ($C(A) = 0,\ f(A) = 5$):

$$\begin{aligned}\text{unit} &= \frac{1{,}288{,}490{,}187}{10} = 128{,}849{,}018\\ \text{low} &= 2{,}147{,}483{,}645 + 128{,}849{,}018 \times 0 = 2{,}147{,}483{,}645\\ \text{range} &= 128{,}849{,}018 \times 5 = 644{,}245{,}090\end{aligned}$$

Renormalization check: range $= 644{,}245{,}090 \geq 16{,}777{,}216$, so continue.

Symbol 3:
Compute offset:

$$\text{offset} = \frac{2{,}789{,}021{,}696 - 2{,}147{,}483{,}645}{644{,}245{,}090} = \frac{641{,}538{,}051}{644{,}245{,}090} \approx 0.9958$$

Find symbol containing 0.9958:

- A: $[0, 0.5)$ — no
- B: $[0.5, 0.8)$ — no
- C: $[0.8, 1.0)$ — yes, contains 0.9958

Decoded symbol: **C**
Update using C ($C(C) = 8,\ f(C) = 2$):

$$\begin{aligned}\text{unit} &= \frac{644{,}245{,}090}{10} = 64{,}424{,}509\\ \text{low} &= 2{,}147{,}483{,}645 + 64{,}424{,}509 \times 8 = 2{,}147{,}483{,}645 + 515{,}396{,}072 = 2{,}662{,}879{,}717\\ \text{range} &= 64{,}424{,}509 \times 2 = 128{,}849{,}018\end{aligned}$$

Renormalization check: range $= 128{,}849{,}018 \geq 16{,}777{,}216$, so continue.

Symbol 4:
Compute offset:

$$\text{offset} = \frac{2{,}789{,}021{,}696 - 2{,}662{,}879{,}717}{128{,}849{,}018} = \frac{126{,}141{,}979}{128{,}849{,}018} \approx 0.9790$$

Find symbol containing 0.9790:

- A: $[0, 0.5)$ — no
- B: $[0.5, 0.8)$ — no
- C: $[0.8, 1.0)$ — yes

Decoded symbol: **C**
Update using C ($C(C) = 8,\ f(C) = 2$):

$$\begin{aligned}\text{unit} &= \frac{128{,}849{,}018}{10} = 12{,}884{,}901\\ \text{low} &= 2{,}662{,}879{,}717 + 12{,}884{,}901 \times 8 = 2{,}662{,}879{,}717 + 103{,}079{,}208 = 2{,}765{,}958{,}925\\ \text{range} &= 12{,}884{,}901 \times 2 = 25{,}769{,}802\end{aligned}$$

Renormalization check: range = 25,769,802 $\geq$ 16,777,216, so continue.

Symbol 5:
Compute offset:

$$\text{offset} = \frac{2{,}789{,}021{,}696 - 2{,}765{,}958{,}925}{25{,}769{,}802} = \frac{23{,}062{,}771}{25{,}769{,}802} \approx 0.8950$$

Find symbol containing 0.8950:

- A: $[0, 0.5)$ — no
- B: $[0.5, 0.8)$ — no
- C: $[0.8, 1.0)$ — yes

Decoded symbol: **C**
Update using C ($C(C) = 8$, $f(C) = 2$):

$$\text{unit} = \frac{25{,}769{,}802}{10} = 2{,}576{,}980$$
$$\text{low} = 2{,}765{,}958{,}925 + 2{,}576{,}980 \times 8 = 2{,}765{,}958{,}925 + 20{,}615{,}840 = 2{,}786{,}574{,}765$$
$$\text{range} = 2{,}576{,}980 \times 2 = 5{,}153{,}960$$

Renormalization check: range = 5,153,960 $<$ 16,777,216 — **renormalization triggered!**

Renormalization During Decoding

When range $< 2^{24}$ during decoding, we need to read the next byte from the compressed stream and incorporate it into the code. This is the inverse of what the encoder did.

Understanding the Symmetry:
During encoding renormalization:

1. Output the most significant byte of low (bits 31-24)
2. Shift low left by 8 bits (bits 23-0 become bits 31-8, bits 7-0 become 0)
3. Shift range left by 8 bits

During decoding renormalization, we need to reverse this process:

1. Read the next byte from the stream (the byte that the encoder output)
2. Shift code left by 8 bits to make room for the new byte
3. Insert the new byte into the least significant position (bits 7-0) of code
4. Shift low left by 8 bits (same as encoder)
5. Shift range left by 8 bits (same as encoder)

The key insight is that both encoder and decoder shift low and range left by 8 bits. The difference is that the encoder **outputs** the byte that falls off the left side, while the decoder **reads** that same byte and inserts it on the right side of code.

Our Example:
Before renormalization (after decoding symbol 5):

$$\begin{aligned} \text{code} &= 2{,}789{,}021{,}696 \quad (\text{0xA6591C00}) \\ \text{low} &= 2{,}786{,}574{,}765 \quad (\text{0xA61A2FAD}) \\ \text{range} &= 5{,}153{,}960 \end{aligned}$$

We read the next byte from the stream (the fifth byte: 0x00):
Step 1: Shift and update code

$$\begin{aligned} \text{code} &\leftarrow (\text{code} \ll 8) \ \& \ (2^{32} - 1) \\ &= (2{,}789{,}021{,}696 \ll 8) \ \& \ \text{0xFFFFFFFF} \\ &= \text{0x591C0000} = 1{,}494{,}994{,}688 \end{aligned}$$

Now insert the new byte into the least significant position:

$$\begin{aligned} \text{code} &\leftarrow \text{code} \mid \text{next_byte} \\ &= 1{,}494{,}994{,}688 \mid 0 = 1{,}494{,}994{,}688 \end{aligned}$$

Step 2: Shift low

$$\begin{aligned} \text{low} &\leftarrow (\text{low} \ll 8) \ \& \ (2^{32} - 1) \\ &= (2{,}786{,}574{,}765 \ll 8) \ \& \ \text{0xFFFFFFFF} \\ &= \text{0x1A2FAD00} = 439{,}463{,}680 \end{aligned}$$

Step 3: Shift range

$$\text{range} \leftarrow \text{range} \ll 8 = 5{,}153{,}960 \times 256 = 1{,}319{,}413{,}760$$

Now range $= 1{,}319{,}413{,}760 \geq 16{,}777{,}216$, so stop renormalization.

Symbol 6:
Compute offset:

$$\text{offset} = \frac{1{,}494{,}994{,}688 - 439{,}463{,}680}{1{,}319{,}413{,}760} = \frac{1{,}055{,}531{,}008}{1{,}319{,}413{,}760} = 0.8$$

Find symbol containing 0.8:

- A: $[0, 0.5)$ — no
- B: $[0.5, 0.8)$ — note: B is $[0.5, 0.8)$, which **excludes** 0.8
- C: $[0.8, 1.0)$ — yes, contains 0.8 (the left endpoint is included)

Decoded symbol: **C**

Decoding Complete

We have recovered the original sequence:

$$\boxed{\texttt{B A C C C C}}$$

Decoding State Table

Step	Code	Low	Range	Symbol
Start	2,789,021,696	0	4,294,967,296	—
1	2,789,021,696	2,147,483,645	1,288,490,187	B
2	2,789,021,696	2,147,483,645	644,245,090	A
3	2,789,021,696	2,662,879,717	128,849,018	C
4	2,789,021,696	2,765,958,925	25,769,802	C
5	2,789,021,696	2,786,574,765	5,153,960	C
Renorm	1,494,994,688	439,463,680	1,319,413,760	—
6	1,494,994,688	439,463,680	1,319,413,760	C

Key Observations

1. **Decoding mirrors encoding:** The decoder uses the same cumulative frequencies and update formulas as the encoder.
2. **Renormalization symmetry:** The encoder outputs a byte when range $< 2^{24}$; the decoder reads that same byte when range $< 2^{24}$ and incorporates it into code. Both update low and range identically (shifting left, which introduces zeros on the right).
3. **Why code gets the byte, not low:** The encoder's low gets zeros when shifted left because those bits are not yet determined. The decoder's code needs the next byte from the stream because that byte represents the next part of the encoded number that was output by the encoder.
4. **Offset calculation:** The offset determines which symbol's interval contains the current code value. The symbol is found by comparing the scaled offset with cumulative frequencies.
5. **Precision maintenance:** Renormalization ensures that range remains large enough to maintain precision throughout the decoding process.

1.10 Direct Comparison: Arithmetic vs. Range Coding

The following table summarizes the parallel between the two methods:

Concept	Arithmetic Coding	Range Coding
Representation	Real numbers in $[0, 1)$	Integers in $[0, R)$
Initial state	low $= 0$, high $= 1$	low $= 0$, range $= R$
Unit size	range $=$ high $-$ low	unit $=$ range/total
Low update	low $+$ range $\times C(s)$	low $+$ unit $\times C(s)$
Range update	range $\times p(s)$	unit $\times f(s)$
Precision handling	Floating-point normalization	Integer renormalization
Final code	Midpoint of final interval	Integer from final interval

1.11 Summary

- Arithmetic coding uses real-number intervals and is mathematically elegant.
- Range coding uses integer intervals but produces the same final interval.
- Range coding is faster in practice because it avoids floating-point arithmetic and uses only integer operations.
- Both achieve compression close to the theoretical entropy limit.
- The two methods are mathematically equivalent; range coding is simply an integer-based implementation of arithmetic coding.
- Renormalization in range coding is integrated into the symbol encoding loop, outputting bytes when the range becomes too small.
- Decoding mirrors encoding, using the same cumulative frequencies and update formulas to reconstruct the original message.
- During renormalization, both encoder and decoder shift low and range left (introducing zeros). The encoder outputs the byte that falls off, while the decoder reads that byte and inserts it into code.

1.12 Exercises for Students

1. Encode the message `CAB` using arithmetic coding with the same probabilities. Show each step.
2. Encode the same message `CAB` using range coding with $R = 65536$ and compare the final intervals.
3. Explain why range coding is more suitable for hardware implementation than arithmetic coding.
4. What would happen if we used a smaller initial range, such as $R = 1000$, for range coding? How would it affect compression quality?
5. Write pseudocode for an arithmetic coding encoder and a range coding encoder. Identify the key differences.
6. Using the figures provided, explain how the interval refinement process differs between arithmetic coding and range coding.
7. Why is the update order (first high, then low) important in arithmetic coding?
8. What is the purpose of renormalization in range coding? In the normalization example, why did we need to renormalize after encoding the fifth symbol?
9. Modify the normalization example to encode the message `B A C C` instead. Would renormalization still be triggered? Why or why not?
10. Using the decoding algorithm, decode the compressed output from Exercise 2 (the `CAB` message) to verify you recover the original message.

11. Explain why the decoder shifts code left and inserts the next byte, while the encoder shifts low left and outputs the byte that falls off. What is the symmetry?